

For segment registers only these types of **MOV** are supported:

- MOV SREG, memory
- MOV memory, SREG
- MOV REG, SREG
- MOV SREG, REG

Micro Controller

Abacus → first calculator

Micro, transistors

4004 → 4 bits

VonNeuman machines

bit count

Binary digit

Real memory: 1MB

machine language

Assembly

16bit x 8086 : 20 address pin

nibble, byte, bit

CISC

RISC

The micro processor (CPU, MP)

data bus

Address bus

Control bus

• MADC

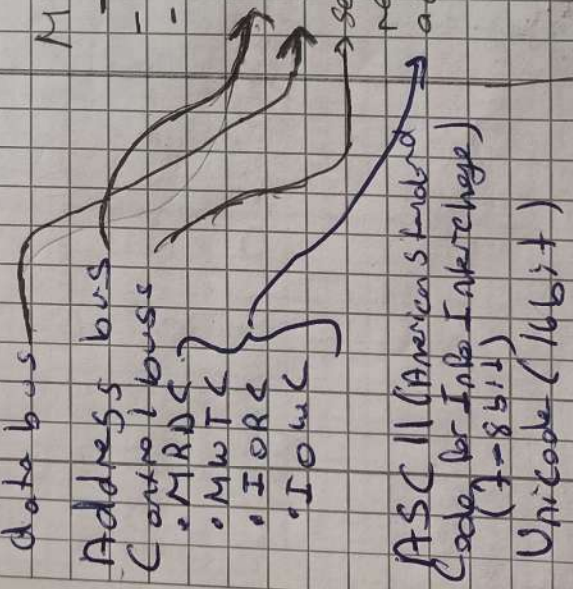
• MWT

• IORC

• IOWC

ASCII (American Standard Code for Info Interchange)
(7-8 bit)

Unicode (16 bit)



MP's 3 main task

- data transfer between I/O and memory I/O
- simple arithmetic or logic operations
- program flow via

Requests a memory location from the memory or I/O
transfers info between MP and its manager I/O
selects and causes memory or I/O devices to perform
reader write methods.

active low when clock signal goes 1 to 0

CPU's bit count; determines the task count in one clock

I/O - CPU - Memory

put address to the AB → select I/O Device, put data to DB → send data to I/O

Eniac: First programable, general purpose computer

Micro: Comes from transistors

4004: Intel's first processors

Background: The number of bits:

- determines the number of addresses it can
- determines the amount of info it can process once.

Assembly first human readable 'pl'.

8086 have 20 address pins

I/O port address similar to a memory address
128 k nibble

CISC → building easy
→ programming hard

RISC → building hard
→ programming easy

Memory and I/O controlled via instructions stored in memory, etc by the MP

Word = 2 bytes
 Double = 4 bytes

+85442 =
 [] [] [] []
 +85442 x 10²
 Significance
 Significant

Pipelining

Register = y02mag

In little-endian → puts least significant number first (x8086)
 In big-endian → puts most significant number first.
 x8086 → DD takes four bytes
 Real number divided it to its component to store the data

Register

x8086 → data bus 16 bit / x8086 → address bus 20 bit

Registers are like hands of the MP. It stores the data temporarily while execution makes some operations on these data.

BIU (Bus Interface Unit)

EU (Execution Unit)

Registers:
 Program visible
 Program invisible

Register: y02mag

AX = 8 bit, 16 bit, 32 bit

AX AL

Flags

C (Carry)

Parity (odd/even)

A (auxiliary)

S (sign)

T (trap)

I (interrupt)

D (direction)

O (overflow)

16 bit ALU → Arithmetic Logic Unit

Flags → Holds the condition of ALU after the execution

6-byte Instruction Queue → holds next instructions (first in first out)

16 bit Registers

AX, BX, CX, DX low and high

SI, DI, SP → addressing only

Flags → whenever AL performs operation corresponding flag gets set or clear

IP → Instruction Pointer

Segment registers (CS, DS, ES, SS) → RM, PM, flat memory

AX → auxiliary carry

1011 1001

1111 1000

1011 0001 Parity = 1

Overflow → sign

Every CPU has its own ALU

Register File

Data Pointer AND Index Regs 8w

16 bit ALU

Flags

6-8 Byte Instruction Queue

Control & Timing

Compiler → assembler → linker → loader
 (assembly) (object file) (executable) (loads to the memory)

do register

begin

catch

(RIP) advance PC

decode

execute

end

AX → A for accumulator & arithmetic

BX → Base, is used for addressing

CX → counter, loop counter (CX times)

DX → Data, general purpose data

BP → Base pointer → points to a memory location

DI → Destination Index → string destination data

For the string instructions if diff to the target

SI → Source Index → string source read

Instruction Pointer (RIP) → points next instruction in a memory

ESP → stack pointer → an area of memory called stack

Keelin Color

MOV VARI, 44H

Segment Registers
CS (code) → Segment holds code (program code)
DS (data) → most data used by program
ES (Extra) → additional segment to hold destination data
SS (Stack) → area of memory for stack (BP, SP and stack segment)
Real Mode Memory Addressing
uses Segment and offset address
Each segment 64KB allows relocatable data/program
(not in 64 bit, direct address)

- Addressing Modes -

Data Addressing
Program memory Addressing
Stack memory Addressing

MOV CS, anything
is false

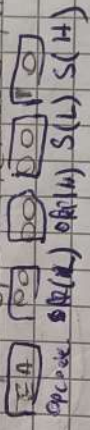
it for beta
F for binary
if have starting with
letter put 0 to beginning

MOV AX, BX
destination ← source

Register	MOV AX, BX	Register BX	→ AX
Immediate	MOV CH, 3AH	Data 3AH	→ Register CH
Direct	MOV [1234H], AX	AX	→ DS:10H + DISP → Memory address 1000H + 1234H
Register Indirect	MOV [BX], CL	Register CL	→ DS:10H + L BX → 10300H
Base-plus-index	MOV [BX+SI], BP	BP	→ DS:10H + BX + SI → 10500H
Register relative	MOV C, [BX+4]	1030H	→ C1
Base-Relative	MOV [BX+SI], DX	DX	→ DS:10H + [BX+SI] → 11500H
-plus-index			1000H + 1000H + 0300H + 0200H

- Program Addressing Modes -

- mainly MP used for jump and call's
- not often as relative and indirect
- The opcode includes the address for storage



Relative to the instruction pointer (IP)
1 byte for instruction 1 or 2 bytes for displacement (0+2+1)
Indirect uses registers
Stack Memory Addressing
push, pop (instructions) 2 byte value never 1 byte

general purpose registers } full back up of registers
→ flag register

Segment offset

CS → Content of R

CS - IP - Instruction A.
SS - SP or BP - Stack A.
DS - DX, DI, SI or RDI - Data A.
ES - DI for string instructions - String Destination A.

push → buffer/stack → 8 bit value CS

Data Movement Instructions



- Selects the operation performed by the CPU
- First 6 bits of the first byte of the binary opcode
- Register or memory (where data flows)
- On byte level or word level



MOD Field

Specifies addressing mode.

- If MOD = 11, register addressing mode is selected
- If MOD contains 00, 01, or 10, the R/M field directs

00H - 4FH (positive)

000H - 004FH

80H - FFH (negative)

FF80H - FFFFH

Orig 100h

assembler directive
(file will be loaded at)
(the address 100h)

All 8 bit displacements are sign-extended into 16 bit instructions when the processor executes the instructions

REG (Register Assignments)

which register to use

R/M Memory Addressing

If the MOD contains 00, 01, 10 the R/M field takes on a new meaning.

00 - 00 displacements [DI]

01 - for 8 bit displacement

11 - R/M also indicates a register R/M = 00 (SP)

Variables

Name DB value (8 bit) -> define byte
Name DW value (16 bit) -> " word

Arrays

Array-Name Data-Type Value (declare array)

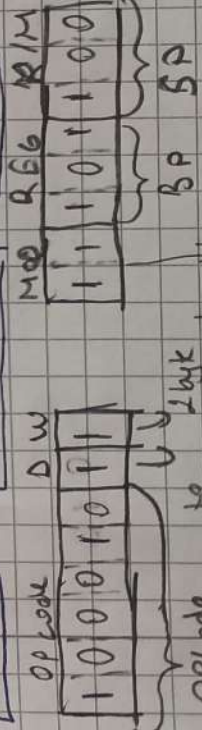
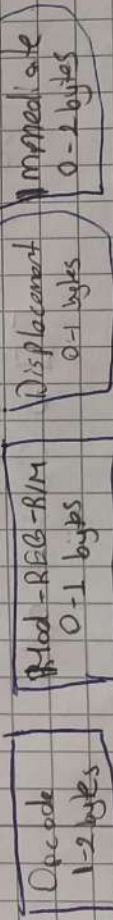
Array-Name Data-Type Number of elements Dup(?)

how many?

We can access any index using [BX, 3] or [MOV BX, 3] [BX]

DW cannot be used to declare strings

LFA -> to take the offset address of a variable
Constants, EQU -> to declare constants (true) name EQU <?>



OP code to 2 byte word

Register

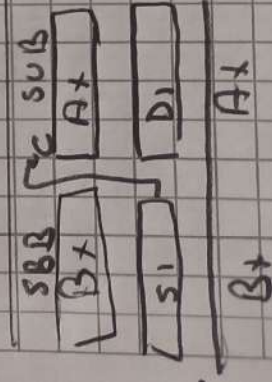
R/M is

register

do forever
Fetch
advance PC
decode
execute
end

Arithmetic and Logic Instructions

Flags	ADD
Carry	$AX = AX + BX$
Zero	$ADC\ AX, BX \Rightarrow AX = AX + BX + C$
Sign	
Overflow	Segment registers and Memory-to-memory not allowed
Parity	
Auxiliary Carry	SUB
Interrupt	$SUB\ AX, BX \Rightarrow AX = AX - BX$
Direction	CMP
Trap	$CMP\ AX, BX \Rightarrow AX - BX$ not store result look for flags
	MUL / IMUL (s)
	$AX = AX * \text{byte}$ $DX:AX = AX * \text{word}$
	DIV / IDIV (s)
	$ \begin{array}{r} AX \text{ byte} \\ \hline \text{an} \end{array} $ $ \begin{array}{r} DX:AX \text{ word} \\ \hline DX \end{array} $



Program Control Instructions

JMP label

label must start with letters and end with ":"

JMP:

Short jump: $EA \text{ Displacement} + IP/128$ } relative jumps
 Next jump: $EA + 2$ }
 For jump: $EA + IP$ } we can change segments.
 Low High L. H.

Assembler calculates the jump instructions displacement value when converts to machine code.

Conditional Jumps (are always short jumps)

For signed numbers

JE, JZ Equal or zero $ZF=1$

JG, JNL greater $ZF=0$ and $SF=0$

not less or equal

JL, JNG less $SF \neq 0$

not greater or equal

JLE, JNGE less or equal $ZF=1$ or $SF \neq 0$

not greater

For unsigned

use above and below look for C not S

Loop label

special jump until $CX=0$

Loope (while equal) $CX \neq 0$ and condition is true

LoopNE $CX \neq 0$ while condition is not equal

LoopNZ $CX \neq 0$ and $ZF=0$ loop if $CX \neq 0$ and $ZF=1$

JCXZ jump to label if CX is zero

Procedure Syntax

name Proc

function/

method

RET

name END P

use Call to call a procedure

pos parameters using regs or stack

All conditional jumps have one big limitation, unlike jmp instruction they can only jump 128 bytes forward and 128 bytes backward

MEMORY INTERFACE

Address Connections
Memory devices have address inputs to select memory location within the device

- The number of address pins on a memory device is determined by the number of memory locations it can store.
- The number of memory locations can be extrapolated from the number of pins.

Data Connections

- All memory devices have a set of data outputs or inputs/outputs.
- Data connections are points at which data are entered for storage or extracted for reading.

- An 8-bit wide memory device is often called a byte-wide memory.

- Catalog listings of memory devices often refer to memory locations times bits per location.

Selection Connections

- Each memory device has an input that selects or enables the memory device.
- RAM has at least one $\overline{CS}$ or $\overline{S}$ input, and ROM has at least one $\overline{CE}$.

Control Connections

ROM: $\overline{OE}$ $\rightarrow$ output enable or gate connection, which allows data flow from output data pins. Latches & buffers needed to read data.

RAM has one or two control inputs, it is one called $\overline{RW}$.

If the RAM has two control inputs, they are usually labeled as $\overline{WE}$ (write) and $\overline{OE}$ (read).

Both

- Both controls must never be active at the same time.

- If both inputs are inactive, data are neither written nor read (the connections are at their high-impedance state).

ROM Memory

non-volatile memory

EPROM $\rightarrow$ Erasable Programmable Read-Only Memory. When software must be changed often or ROM is uneconomical. (depending on type available if exposed to UV).

PROM $\rightarrow$ Programmable ROM not common today, once programmed cannot be erased.

RAM $\rightarrow$ Read mostly memory (Flesh memory), electrically erasable but does not time then RAM.

- used to store skip info's for systems such as the video card in computer.

1K x 8 $\rightarrow$ 1KB

32K x 16 $\rightarrow$ 64KB

$\overline{CS}$ $\rightarrow$ chip select

$\overline{CE}$ $\rightarrow$ chip enable

$\overline{S}$ $\rightarrow$ select

$\overline{OE}$ $\rightarrow$ output enable

$\overline{WE}$ $\rightarrow$ write enable

RAM types:

EEPROM

EAROM $\rightarrow$ electrically erasable

NOVRAM (non-volatile RAM)

Memory Interface

SRAM retains data as long as DC power applied (Volatile)
- Called static because no special action is required to retain data } integrated with transistor.
- So fast so used in caches

DRAM: essentially same as SRAM but needs refresh (rewritten) after 2, 4 ms
integrated with capacitors.

DRAMs is often places SIMMs or DIMMs. requires so many address pins

DDR RAM: double data rate ram

SIMM

DIMM

SIMM → single in-line memory modules

DIMM → dual " " " "

SRAM vs DRAM

SRAM is static while DRAM is dynamic

SRAM is faster compared to DRAM

SRAM consumes less power than DRAM

SRAM uses more transistors per bit of memory

SRAM is more expensive

DRAM used in main memory and SRAM

used in cache memory

I/O Interface

The I/O Instructions

OUT: transfer information to an I/O device

IN: reads from an I/O device

Instructions are also provided to transfer strings of data between memory and I/O

For data transfer between MP and I/O PL, AX, BX

For I/O address DX for 16 bit or (ports) directly binary number for byte

The 16 bit address is called a variable address because it is stored in DX

- I/O ports are 8 bits in width

- all bit is actually 2 consecutive 8 bit ports being addressed

- In transfer port number appears in address bus, I/O interface decodes the port number in the same manner as a memory address,

Isolated (direct) and Memory Mapped I/O

The PC does not use memory-mapped I/O

Isolated I/O

Addresses for isolated I/O's called ports, are separate from memory

Advantage user can expand the memory to its full size disadvantage is must use in, out etc.

Memory Mapped I/O

does not use in, out instructions. It uses any instruction that transfer data between the MP and memory

Disadvantage some part of memory used for I/O map

Handshaking

A method of the control called handshaking or polling, synchronizes the I/O device with MP. (MP much faster)

The software polls or tests the READY pin to decide whether the pin is busy

I/O Communication Methods

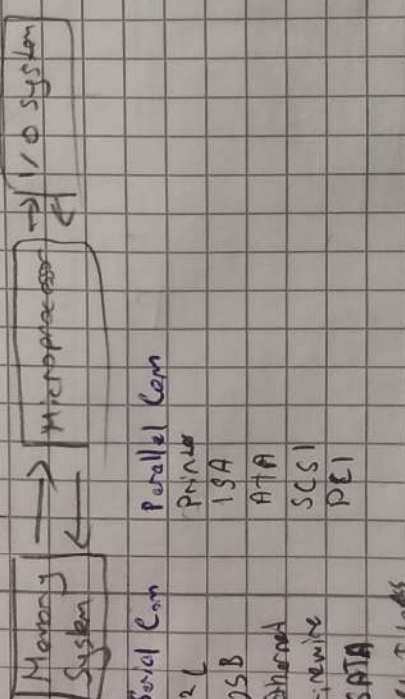
Serial communication is the process of sending data one bit at a time, sequentially. Parallel communication, several bits are sent together on a link comprising.

Serial vs Parallel

- Serial simple and cheap, parallel complex and expensive.

- Parallel sends at once but not receives at once because of the physical limitations.

- So, where certain speed and distance threshold, serial communication is superior.



Serial vs Parallel

Serial

Parallel

simple

complex

cheap

expensive

sends at once

not receive at once

Interrupts

Polling: Ask every device sequentially. It needs service. Note, no device may read ser-

Interrupts during the poll
Interrupts during interrupts CPU to indicate that it needs service

Types of Intercepts

Software interrupt is an interrupt generated within a processor by executing an instruction. Software interrupts are often used to implement sys calls.

Hardware interrupt is an interrupt generated by hardware such as I/O devices.

Hardware interrupt is an interrupt generated by hardware. Software interrupt is an interrupt that may be ignored.

Maskable interrupt is a hardware interrupt that can be ignored. Non-maskable interrupt (NMI) is a hardware interrupt that can never be ignored. Interrupts on Intel

INTR and NMI two request interrupts

RNA polymerase request 1st intron
RNA for acknowledge 1st request requested through 1st ITR

17 Interrupt Rlog)

Intercept Vectors

Interrupt vectors table is located in first 1024 bytes of memory.

- An interrupt vector contains the address of the ISR (segment offset)

Internet Service Routine (ISR) (Interrupt Handler)

- **VS D** combine the routines that is to be executed when a particular interrupt occurs

- The first 5 int vectors are identical in all int CPU's

- The first 3 nt vectors are derived in all runs (100%)
- The first 19 nt vectors (remaining 224 is not evaluable)

Software Interrupt Instructions

general concept in vector spaces

2017 5/3/2017

$\text{R} \rightarrow \text{return for both software and hardware}$

Operation of a Real Mode Interrupt

12 or more are present

1.) Flag register contents are pushed on the stack.

9.) It is cleaned disabling the WTR Pin.

3-1) (S) pushed to stock

4-18, " "

k - Interval vector's address

6.1 when ISR is done pop the stack

44

IF set(1) CPU can receive Interrupt

$\text{E_clocked}(0)$ CPU prevents the INTR pin from causing an interrupt of

Two ways of fraction with 10's

Polling (Handshaking), Interrupt

Intercept vector

Segment H

Sequences

offset 17

02851

Order Per 1 NT

1. Instruction to rule

2-) single step

27.28

6-2) compressed segment available

5-1) WTR

b) Vint instruction is the order presented

<i>Segment</i>	<i>Offset</i>	<i>Special Purpose</i>
CS	IP	Instruction address
SS	SP or BP	Stack address
DS	BX, DI, SI, an 8- or 16-bit number	Data address
ES	DI for string instructions	String destination address

- INTO checks or tests the overflow flag (O).
 - If $O = 1$, INTO calls the procedure whose address is stored in interrupt vector type 4
 - If $O = 0$, INTO performs no operation and the next sequential program instruction executes

- 1. Flag register contents are pushed on the stack.
- 2. Interrupt flag is cleared, disabling the INTR pin.
- 3. Contents of the code segment register (CS) are pushed onto the stack.
- 4. Contents of the instruction pointer (IP) are pushed onto the stack.
- 5. Interrupt vector contents are fetched and placed into IP and CS so the next instruction executes at the interrupt service procedure addressed by the vector.
- 6. When ISR is done, previous status is restored from the stack.

- Catalog listings of memory devices often refer to memory locations times bits per location.
 - a memory device with 1K memory locations and 8 bits in each location is often listed as a $1K \times 8$ by the manufacturer

- These types of operands are supported:
 - MOV REG, memory
 - MOV memory, REG
 - MOV REG, REG
 - MOV memory, immediate
 - MOV REG, immediate